24/10 Métricas en los Diferentes Enfoques de Gestión (debrief)

Resumen Ejecutivo

TÍTULO: La Cuantificación Estratégica en la Ingeniería de Software: Métricas, Enfoques y Mejora Continua.

Las métricas de software son esenciales para cualquier organización que busque mejorar su proceso de desarrollo. Estas se categorizan en tres dominios interrelacionados: **Métricas de Proceso** (estratégicas, para la mejora organizacional), **Métricas de Proyecto** (tácticas, para el control de recursos y riesgos) y **Métricas de Producto** (técnicas, para evaluar la calidad del software construido).

Enfoque Tradicional (Procesos Definidos):

El desarrollo tradicional se basa en procesos definidos que buscan la predictibilidad y la repetición de resultados. Por ello, implementa un amplio abanico de métricas de proceso y proyecto para monitorear el cumplimiento estricto del plan, como la **Volatilidad de Requerimientos**, la **Distribución del Esfuerzo** y la **Precisión de las Estimaciones**. Estas mediciones son cruciales para la planificación y la gestión proactiva de riesgos.

Contrastando con Enfoques Empíricos (Ágil y Lean):

Los enfoques modernos (Ágil y Lean/Kanban) se basan en procesos empíricos, que asumen la complejidad y la naturaleza cambiante del software. La **Regla de Oro Ágil** establece que la medición es una **salida, no una actividad**, y que el **Software Funcionando** es la principal medida de progreso.

- **Métricas Ágiles:** Se enfocan en el producto. La **Velocidad** (cantidad de *Story Points* completados por iteración) y el **Running Tested Features (RTF)** son indicadores clave del avance real del producto.
- **Métricas Lean/Kanban:** Se centran en el flujo. El **Lead Time** (tiempo total de entrega, vista del cliente) es la métrica principal para la planificación y la mejora del flujo de valor.

Métricas Transversales (Producto y Calidad):

Independientemente del enfoque, el producto se mide por su **Tamaño** (ej. PF, LDC, Puntos de Historia) y sus **Defectos** (ej. Cobertura, Densidad de Defectos). Estas métricas se utilizan para garantizar que los artefactos cumplen con la calidad requerida.

Conclusión y Buenas Prácticas:

El éxito de la medición reside en su utilidad práctica, evitando el exceso de burocracia y la ambigüedad. Las métricas deben **informar la mejora** y **nunca ser utilizadas para juzgar** o castigar al personal. La elección del "mejor" enfoque de medición debe ser pragmática y adaptada al contexto específico del proyecto (tipo de sistema, equipo, cultura), ya que no existe un proceso universal.

MÉTRICAS DE SOFTWARE EN LOS DIFERENTES ENFOQUES DE GESTIÓN

Síntesis: Las métricas de software son herramientas de medición continuas aplicadas al proceso de desarrollo y a sus productos para obtener información objetiva y oportuna que fundamente la toma de decisiones, la predicción y la mejora. El dominio de las métricas se divide en tres áreas fundamentales: Proceso, Proyecto y Producto.

Explicación Detallada:

La ingeniería es, por naturaleza, una disciplina cuantitativa. En la ingeniería de software, las métricas son esenciales para informar, motivar, comparar, entender, evaluar, predecir y mejorar.

Una **métrica** se define como la aplicación continua de mediciones basadas en técnicas para suministrar información relevante a tiempo al líder del proyecto. La medición se relaciona con la unidad, que es una cantidad particular definida por convención para comparar magnitudes.

- **Métricas Directas:** Son aquellas que se pueden obtener sin depender de ninguna otra métrica.
  - *Ejemplo:* Líneas de código (LDC) de un módulo o la fiebre.
- **Métricas Indirectas:** Provienen de una función de cálculo cuyos argumentos son otras métricas (directas o indirectas).
  - *Ejemplo:* Total de horas de programación o la eficiencia en la remoción de defectos (ERD).

Dominios de las Métricas de Software:

1. **Métricas de Proceso (Estratégicas):**
   - **Foco:** Mejorar el proceso de software de la organización a largo plazo.
   - **Nivel:** Estratégico y organizacional.
   - **Visibilidad:** Son públicas para toda la organización del software.
   - **Uso/Vínculos:** Permiten una visión profunda de la eficacia del proceso, determinando qué funciona y qué no. Su consolidación se forma a partir de las métricas de proyecto. Son vitales para mejorar el nivel global de madurez del proceso.
   - *Ejemplos:* Errores previos a *releases* por proyecto, Propagación de errores de fase a fase, Desviación organizacional de estimaciones.
2. **Métricas de Proyecto (Tácticas):**
   - **Foco:** Recursos dedicados al proyecto (costos, esfuerzo, tiempo, estimaciones).
   - **Nivel:** Específico del proyecto, responsabilidad del Líder de Proyecto.
   - **Visibilidad:** Privadas, visibles solo para los involucrados.
   - **Uso/Vínculos:** Mejoran la planificación, previenen retrasos, reducen riesgos potenciales y evalúan la calidad del producto en todo momento.
   - *Ejemplos:* Esfuerzo/Tiempo por tarea de Ingeniería de Software, Fechas de entregas reales vs. programadas, Cantidad de cambios y sus características.
3. **Métricas de Producto (Técnicas):**
   - **Foco:** Lo que se construye (artefactos resultantes).
   - **Nivel:** Técnico, responsabilidad del equipo de desarrollo.
   - **Uso/Vínculos:** Generan indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, y la calidad global del software. Buscan garantizar que el producto cumpla con los requerimientos y los criterios de calidad, y que esté libre de errores.
   - *Ejemplos:* Líneas de código (LDC), Puntos de Función (PF), Defectos por severidad.

MÉTRICAS DE SOFTWARE EN EL ENFOQUE TRADICIONAL BASADO EN PROCESOS DEFINIDOS

Síntesis: El enfoque tradicional, basado en procesos definidos, se apoya en una gestión rigurosa y predictiva. Este modelo utiliza una amplia variedad de métricas en los tres dominios (proceso, proyecto y producto) para administrar y controlar la predictibilidad del desarrollo.

Explicación Clara y Comparativa: Proceso Definido vs. Empírico

- **Procesos Definidos (Tradicional):**
  - **Definición:** Asumen que el mismo proceso, repetido una y otra vez, dará los mismos resultados, independientemente del equipo. Están inspirados en las líneas de producción.
  - **Gestión:** La administración y el control provienen de la predictibilidad. Requieren que el proceso esté bien definido, ya que lo definido es repetible, medible y, por lo tanto, mejorable (como requieren modelos de mejora como CMMI, ISO).

- **Énfasis:** Tienden a dar más importancia a la documentación del avance y al cumplimiento de los hitos del proceso, que al avance tangible del software en sí.

- **Procesos Empíricos (Ágil/Lean):**
  - **Definición:** Asumen que el desarrollo es complicado y con variables cambiantes, por lo que la repetición del proceso puede arrojar resultados diferentes. Se ajustan mejor a procesos complejos y creativos.
  - **Gestión:** La administración y el control se basan en la inspección frecuente y la adaptación. El conocimiento se genera a partir de la experiencia (empirismo).
  - **Énfasis:** Requieren un modelo empírico de control de proceso debido a la complejidad e impredecibilidad del desarrollo de sistemas. Se centran en ciclos de inspección y revisión *just in time*, reemplazando la planificación detallada inicial.

Métricas Comunes en el Enfoque Tradicional (Proyecto y Proceso):

Las métricas del proyecto en el enfoque tradicional son fundamentales para mejorar la planificación.

Categoría	Métrica	Vínculo / Riesgo
Tiempo/Schedule	Tiempo Calendario	Seguimiento de plazos y cumplimiento de la Triple Restricción (Alcance, Costo, Tiempo).
Esfuerzo/Costo	Distribución del esfuerzo; Performance actual y planificada de esfuerzo/presupuesto	Utilizadas para estimar (ej: Esfuerzo por KLDC) y para evaluar la eficiencia de las estimaciones.
Gestión de Requerimientos	Status de requerimientos; Volatilidad de requerimientos	Mide el riesgo de cambio en los requerimientos, que es más alto al inicio del proyecto y afecta a la calidad o rendimiento.
Calidad/Defectos	Nro. de defectos encontrados en peer reviews; Defectos en integración y prueba de sistemas	El objetivo es encontrar errores y defectos pronto para minimizar el retrabajo y el costo total del proyecto.

Vínculos con Otras Unidades:

- **Gestión de Proyectos:** Las métricas de proyecto (Esfuerzo, Calendario, Costos) son cruciales para el Líder de Proyecto, quien las usa para cumplir con el contrato y gestionar la relación entre esfuerzo y tiempo.
- **Riesgos:** La administración proactiva del riesgo se utiliza para desarrollar planes de mitigación y contingencia. La **exposición al riesgo** (amenaza total) se calcula como el producto de la probabilidad de ocurrencia y el impacto. Las métricas, como la volatilidad de requerimientos, ayudan a monitorear estos riesgos.

ALGO MÁS SOBRE LAS MÉTRICAS...

Síntesis: Las métricas de software deben ser útiles, prácticas y coherentes con los objetivos del negocio y las restricciones del proyecto, manteniendo siempre un equilibrio entre las necesidades y las expectativas de alcance y calidad.

Explicación Clara: Principios de Medición y Buenas Prácticas

Las métricas deben ser **válidas** (proximidad al valor verdadero o exactitud) y **confiables** (precisión, repetibilidad o reproductibilidad).

Preguntas Clave (Beneficio Práctico):

Antes de implementar una métrica, es fundamental cuestionarse:

1. **¿Nos da más información que la que tenemos ahora?**
2. **¿Es esta información de beneficio práctico?**
3. **¿Nos dice lo que queremos saber?**

El valor de una métrica se valida si es representativa y aumenta la información práctica para el entorno. Es vital sopesar el **costo-beneficio** de definir una métrica, ya que su planificación, ejecución y análisis consume recursos.

Riesgos y Buenas Prácticas:

- **Riesgo de Juzgar:** Las métricas nunca deben usarse para evaluar o castigar a los individuos, sino para entender y mejorar. Si se usan para juzgar, las personas podrían falsear los valores para lograr objetivos, lo cual es perjudicial para el análisis.
- **Mantenlo Simple:** Si se está a millas de distancia del destino, no tiene sentido medir en milímetros. Se recomienda una filosofía minimalista.

Vínculo: Alcance, Calidad y Restricciones:

Las métricas ayudan a **balancear** las Necesidades y Expectativas relacionadas con el Alcance y la Calidad, dentro de las Restricciones del proyecto. Esto se relaciona directamente con la gestión de la **Triple Restricción** (Alcance, Tiempo y Costo/Recursos).

MÉTRICAS DE SOFTWARE EN AMBIENTES ÁGILES PARA PROCESOS EMPÍRICOS

Síntesis: El enfoque ágil, inherentemente empírico, adopta una filosofía minimalista de medición. Su principio rector es que **el software funcionando es la principal medida de progreso**. Las métricas deben ser una *salida*, no una actividad con sobrecarga burocrática.

Regla de Oro Ágil sobre Métricas:

- **La medición es una salida, no una actividad**.
- Nuestra mayor prioridad es la **satisfacción del cliente** mediante entregas tempranas y continuas de software valioso.
- Esto implica medir solo lo que sea necesario y lo que agregue valor para el cliente.

Filosofía y Fundamentos:

- El desarrollo ágil adopta un ciclo de vida **iterativo-incremental** basado en el **empirismo** (conocimiento que proviene de la experiencia y la observación).
- El control se ejerce a través de los tres pilares del empirismo: **Transparencia, Inspección y Adaptación**.
- La medición en Ágil se enfoca principalmente en el **producto** (el incremento entregado), en contraste con el enfoque tradicional que cubre los tres dominios.

Métricas Ágiles Clave:

1. **Velocidad (Velocity):**
   - **Definición:** La métrica más importante del enfoque ágil. Mide la cantidad de **Puntos de Historia** (*Story Points*) completados por un equipo en un *Sprint* (o iteración).
   - **Uso:** Sirve como medida de producto. Los Puntos de Historia no se basan en tiempo absoluto, sino en la complejidad relativa, incertidumbre y esfuerzo.
2. **Capacidad (Capacity):**
   - **Definición:** Métrica listada en el enfoque ágil. Relacionada con la cantidad de trabajo que un equipo puede asumir en una iteración, a menudo basada en la disponibilidad de los recursos.
3. **Running Tested Features (RTF):**
   - **Definición:** Mide directamente los resultados entregados, representando las funcionalidades que están probadas y en funcionamiento.
   - **Análisis:** Es una medición absoluta. Si el gráfico RTF (*Features* vs. *Iteration*) se mantiene constante (llana) o disminuye (pendiente negativa), indica un problema de desarrollo, ya que las características no están incrementando o han dejado de ser válidas.

Vínculos con Otras Unidades:

- **Estimación y Planificación:** El ágil utiliza estimaciones relativas (Story Points). La Velocidad histórica se usa para predecir la cantidad de trabajo que se puede completar en futuros *Sprints*.

- **Despliegue y Calidad:** El concepto de RTF está ligado a la **Integración Continua** (CI) y la **Entrega Continua** (CD), donde el código se prueba e integra continuamente. El software debe estar **potencialmente desplegable** tras cada iteración.

MÉTRICAS EN KANBAN PARA PROCESOS EMPÍRICOS CON ENFOQUE LEAN

Síntesis: Kanban es un enfoque Lean que se centra en el flujo de valor y la eliminación de desperdicios. Sus métricas clave buscan medir la eficiencia del flujo de trabajo y el tiempo que tarda un ítem en moverse desde el inicio hasta la finalización.

Fundamentos Lean y Kanban:

- **Filosofía Lean:** Busca maximizar el valor generado al cliente con el mínimo consumo de recursos. El esfuerzo que no crea valor se considera **desperdicio**.
- **Kanban:** Método para procesos empíricos que implementa los principios Lean. Ambos, Kanban y Scrum, son empíricos y esperan experimentación y adaptación del proceso.

KANBAN: Métricas Clave:

1. **Lead Time (Tiempo de Entrega):**
   - **Perspectiva:** Vista del Cliente.
   - **Definición:** Métrica que registra el tiempo total que transcurre entre el inicio y el final del proceso para un ítem de trabajo. Suele medirse en días de trabajo o esfuerzo.
   - **Uso:** Es la métrica por defecto para la planificación y la mejora del proceso en Kanban. En planificación, se utiliza la previsión probabilística (ej. método Monte Carlo) para proyectar intervalos de fechas de terminación basados en datos históricos de *Lead Time*.
2. **Cycle Time (Tiempo de Ciclo):**
   - **Perspectiva:** Vista Interna.
   - **Definición:** Es una medición más mecánica de la capacidad del proceso. Es similar al *Lead Time*, registrando el tiempo entre el inicio y el fin del proceso (a menudo medido en días de trabajo/esfuerzo).
   - **Alternativa:** También se conoce como Ritmo de Terminación.
3. **Touch Time:** Métrica listada en el enfoque Lean. (No encontrado en fuentes como concepto desarrollado).
4. **Eficiencia Proceso (Flow Efficiency):** Métrica listada en el enfoque Lean. Relacionada con la eficiencia de flujo.

Riesgos y Buenas Prácticas (Lean):

- **Riesgos (Desperdicio):** Se deben identificar y eliminar desperdicios, como el proceso extra (pasos que no aportan valor) o la búsqueda excesiva de información.
- **Buena Práctica (Compromiso):** El principio Lean de "Aplazar el compromiso" se relaciona con dejar abiertas las opciones de diseño para tomar decisiones irreversibles lo más tarde posible.

¿Y PARA EL PRODUCTO SOFTWARE, QUÉ MÉTRICAS?

Síntesis: Las métricas de producto son aplicables a todos los enfoques de gestión (tradicional, ágil, Lean) y se centran en atributos internos (tamaño, complejidad) y externos (calidad, defectos) del software.

Métricas de Producto - TAMAÑO:

El tamaño es una métrica de producto que puede normalizar las medidas de calidad y productividad.

- **Líneas de Código (LDC):** Medida directa. Son poco aceptadas porque dependen del lenguaje de programación y de la capacidad lógica del programador. Es una medida orientada a tamaño.
- **Requerimientos, Features, Clases:** Elementos contables del producto.

- **Casos de Uso (CU) por Complejidad / Puntos de Historia (Story Points):** Medidas de tamaño basadas en la funcionalidad o complejidad.
- **Puntos de Función (PF):** Métrica orientada a la función. El cálculo se basa en las características del dominio de información y la complejidad, y es independiente del lenguaje de programación.

Métricas de Producto - DEFECTOS:

Las métricas de defectos son necesarias en todos los ambientes.

- **Cobertura (Coverage):** Busca cubrir el 100% del análisis de defectos.
- **Defectos por Severidad:** Mide la cantidad de defectos clasificados según su impacto.
- **Densidad de Defectos:** Mide cuántos defectos se encuentran por unidad de tamaño, por ejemplo, por historia de usuario.

Métricas Orientadas al Tamaño (Ejemplos Normalizados):

Estas métricas provienen de la normalización de la calidad y/o productividad respecto al tamaño del software producido.

- Errores por KLDC (miles de líneas de código).
- Defectos por KLDC.
- Esfuerzo por KLDC.
- LDC por persona/mes.

PARA PENSAR... (Reflexión y Conclusión Metodológica)

Síntesis: Es esencial utilizar las métricas para obtener conocimiento y mejorar, evitando usarlas para juzgar a las personas o los enfoques. La elección del mejor enfoque (ágil, tradicional, Lean) depende del contexto y del tipo de sistema que se está desarrollando.

Reflexiones Clave:

- **Compara herramientas para entender no para juzgar**. La intención de las métricas de proceso es mejorar el proceso, no castigar.
- **¿Cuál es mejor?**. No existe un proceso ideal que sirva para cualquier tipo de proyectos.
- **Contexto de Aplicación:** El equilibrio entre un enfoque basado en un plan (tradicional) y uno ágil depende del tipo de software que se va a elaborar, las capacidades del equipo de desarrollo y la cultura de la compañía. Por ejemplo, los sistemas críticos de seguridad requieren un proceso muy estructurado (definido), mientras que los sistemas empresariales con requerimientos cambiantes funcionan mejor con un proceso menos formal (ágil).

Secciones Transversales

Glosario Crítico

Concepto	Definición y Fuente	Vínculo
Métrica	Aplicación continua de mediciones para suministrar información relevante a tiempo.	Se divide en dominios de Proceso, Proyecto y Producto.
Proceso Definido	Asume que la repetición del proceso ofrece resultados iguales, inspirándose en líneas de producción.	Base del Enfoque Tradicional, enfocado en la predictibilidad y documentación.
Proceso Empírico	Asume variables cambiantes y que la repetición puede dar resultados diferentes; basado en la experiencia, inspección y adaptación.	Base de los Enfoques Ágiles y Lean, ajustándose a la complejidad.
Velocidad (*Velocity*)	Métrica de producto ágil que mide la cantidad de Puntos de Historia completados en una iteración.	Uso principal en la planificación de *Sprints* en marcos como Scrum.

Concepto	Definición y Fuente	Vínculo
Punto de Historia (*Story Point*)	Unidad de medida relativa utilizada en Ágil para estimar el tamaño (complejidad, incertidumbre, esfuerzo) de una *User Story*, no basado en tiempo.	Reemplaza las estimaciones absolutas de tiempo, que conducen a errores.
Lead Time	Métrica clave de Kanban (vista del Cliente), registra el tiempo desde el inicio hasta el fin del proceso para un ítem de trabajo.	Fundamental para la previsión probabilística en la gestión Lean.
Running Tested Features (RTF)	Medida directa del progreso en Ágil, representando las funcionalidades probadas y en funcionamiento.	La curva de avance del software, que indica si el producto está creciendo y funcionando.
Defecto	Falla que se identifica en una etapa *posterior* a aquella donde se originó (problema mayor, se trasladó).	Contrasta con el **Error**, que es una falla identificada en la misma etapa donde se originó.

Cuadros Comparativos

Característica	Enfoque Tradicional (Definido)	Enfoque Ágil (Empírico)
Base Metodológica	Procesos definidos, predictibilidad, documentación formal.	Procesos empíricos, adaptación continua, iteración rápida.
Triple Restricción	Alcance fijo; Tiempo y Recursos variables (se ajustan al alcance).	Tiempo y Recursos fijos; Alcance variable (se ajusta al tiempo/recursos).
Medición Principal	Proceso y Proyecto (Costos, Esfuerzo, Tiempo, Volatilidad).	Producto (Software Funcionando, Velocidad, RTF).
Unidad de Estimación	LOC, Puntos de Función, Horas-hombre (absoluta).	Story Points (relativa).
Requerimientos	Especificación *Up Front* (al inicio), alto impacto de errores iniciales.	Relevados continuamente, responden a la variabilidad y retroalimentación.

Métrica	Definición	Enfoque (General)
Esfuerzo	Tiempo/trabajo dedicado por tarea de ingeniería de software.	Tradicional (Proyecto).
Velocidad	Puntos de historia completados por iteración.	Ágil (Producto).
Lead Time	Tiempo total desde inicio hasta entrega al cliente.	Kanban/Lean.
RTF	Funcionalidades probadas y en funcionamiento.	Ágil (Producto).
Densidad de Defectos	Defectos encontrados por unidad de tamaño (ej: por historia de usuario).	Producto (Universal).

Checklist de Buenas Prácticas en Medición

Las siguientes prácticas son cruciales para un programa de métricas exitoso que aporte valor y fomente la mejora:

- **Alineamiento a Objetivos:** Las métricas deben activarse mediante metas empresariales y técnicas claras.
- **Simplicidad:** Mantener el conjunto de métricas simple y fácil de calcular; evitar enterrarse en datos.
- **No Juzgar:** Nunca usar métricas para evaluar, amenazar o castigar a individuos o equipos.
- **Definición Clara:** Establecer qué se va a medir (proceso, producto, proyecto) y definir la forma de calcular la métrica de manera clara y no ambigua.
- **Retroalimentación Continua:** Proporcionar *feedback* regular a quienes recopilan las métricas y utilizarlas para guiar al proyecto o al proceso.
- **Uso Consistente:** Asegurarse de que las métricas se recopilan de la misma manera en todos los proyectos para poder realizar comparaciones válidas.
- **Balance:** No obsesionarse con una sola métrica, sino utilizar un conjunto limitado que proporcione una visión integral (evitar el monocultivo de métricas).

- **Enfoque en Calidad:** Dirigir las métricas a mantener en un mínimo absoluto la propagación de errores entre actividades y la cantidad de defectos liberados.

Mapa de Relaciones Conceptuales (Vínculos con Otras Unidades)

Unidad Relacionada	Concepto de Métrica/Medición	Vínculo Detallado
Gestión de Proyectos	Métricas de Proyecto (Esfuerzo, Calendario, Costo).	Permiten monitorear la **Triple Restricción** y tomar decisiones tácticas para ajustar el flujo de trabajo y el enfoque técnico.
Estimación	Puntos de Función, LDC, Story Points, Datos Históricos.	Las métricas de tamaño y productividad son insumos clave para los modelos de estimación empíricos y para contrastar la precisión de las predicciones.
Riesgos	Volatilidad de Requerimientos, Exposición al Riesgo (P x I).	Las métricas son cruciales en la **Monitorización del Riesgo**, valorando la probabilidad de que un riesgo se vuelva más o menos probable conforme avanza el proyecto.
Calidad de Software (SQA/QC)	Cobertura, Densidad de Defectos, Defectos en *Peer Reviews*.	Las métricas son la base para el **Aseguramiento de Calidad** (QA), permitiendo valorar la calidad de los productos y procesos en tiempo real. Las métricas de revisión (ej. defectos en *peer reviews*) buscan eliminar errores antes de que se propaguen.
Procesos de Desarrollo	Métricas de Proceso, Lead Time/Cycle Time.	Se utilizan para medir la madurez (CMMI/ISO) y mejorar la eficacia de los procesos. Lean/Kanban usa el *Lead Time* para optimizar el flujo de valor.
Despliegue	Running Tested Features (RTF), Lead Time.	En entornos de **Continuous Delivery** (CD), el RTF es una confirmación de que el software entregado en incrementos funciona. El *Lead Time* mide la rapidez de la respuesta al mercado.